大家好，我是 Guide。在 LLM 应用开发中，Prompt 的质量直接定义了 AI 输出的“确定性”。很多小伙伴在写 Prompt 时比较随意，导致模型输出像“开盲盒”，完全靠运气。

本文以本项目中的**简历分析**场景为例，拆解如何通过**结构化设计**让 Prompt 从“能用”进化到“好用”。文中的优化后示例与仓库内 `app/src/main/resources/prompts/resume-analysis-system.st`、`resume-analysis-user.st` 保持一致，可直接对照源码。**Prompt 工程没有银弹，好的指令都是在实战中磨出来的。**

问题：简单 Prompt 的局限性

先看一个典型的“学生思维”版本，这种 Prompt 在 Demo 阶段或许能跑通，但在生产环境下就是“灾难”：

```
请分析以下简历内容，按照要求给出评分和改进建议：

---简历内容开始---
{resumeText}
---简历内容结束---

请严格按照 JSON 格式输出分析结果。
```

这种“意识流”写法存在四大硬伤：

维度	具体表现	带来的后果
角色缺失	模型不知道以什么身份回答，输出风格不稳定。	一会儿像 HR，一会儿像初级开发，专业度忽高忽低。
标准模糊	“评分”依据是什么？满分多少？权重如何分配？	评分随缘，无法作为业务逻辑的判断依据。
输出不可控	“JSON 格式”具体包含哪些字段？结构如何？	经常导致后端解析报错（如 Missing fields 或格式非法）。
质量无保障	没有约束条件，容易出现 AI 幻觉或编造内容。	建议空洞、口水话多，甚至会脑补候选人没做过的项目。

在真实业务场景中，这种不确定性会导致用户体验极差，甚至直接拖垮整条产品线。

解决方案：结构化 Prompt 设计框架

为了让模型乖乖听话，我总结了一套**结构化设计框架**。你可以把它理解为给 AI 下达的一套 **SOP（标准操作规程）**。

其核心构成如下：

- **# Role** → 定义身份与能力边界
- **# Task** → 明确核心任务
- **# Standards** → 建立量化评估标准
- **# Workflow** → 规定执行流程
- **# Constraints** → 设置硬性约束
- **# Output** → 规范输出协议

下面逐一分析每个模块的设计要点。

Role：建立专家人设

优化前：无

优化后：

```
# Role
你是一位拥有 10 年以上经验的资深技术架构师、工程管理专家及高级技术人才顾问。你具备跨语言（Java, Go, Python, Rust, Frontend, Infrastructure 等）的深度技术视野，擅长从底层架构、工程效率和业务价值三个维度对简历进行“穿透式”审计。
```

不要只写“AI 助手”，要给它具体的人设。通过“10 年经验”、“技术架构师”这些关键词，可以有效激活模型权重中与资深工程师相关的知识库。

Task：明确核心目标

优化前：

```
请分析以下简历内容，按照要求给出评分和改进建议
```

优化后：

```
# Task
请对用户提供的简历内容进行深度技术审计、多维度评分，并提供极具实操性的改进建议，特别是针对"项目经历"的重写与优化。
```

动词要精准。“审计”比“分析”更具专业感，同时明确指出“项目经历重写”是重点，引导模型集中“算力”解决核心痛点。

Standards：量化评估标准

优化前：无（模型自由发挥）

优化后：

```
# Scoring Rubrics (Total: 100)
1. projectTechDepth (0-40 分)：是否避开了烂大街的项目，是否体现复杂问题排查或成熟中间件的深度运用，是否有量化产出...
2. skillMatchScore (0-20 分)：技术栈专业度，区分"了解/熟悉/熟练掌握"...
3. contentScore (0-15 分)：模块顺序是否合理...
4. structureScore (0-15 分)：技术名词大小写必须绝对规范（如 Java, Spring Boot, MySQL）...
5. expressionScore (0-10 分)：语言是否简洁...
```

量化！量化！还是量化！ 明确权重分配（项目深度占 40% 是行业共识），能让模型在评分时有据可依，减少主观偏差。

Workflow：规定执行流程

优化前：无

优化后：

```
# Audit Workflow
1. 名词纠错：扫描全文，列出所有不规范的技术名词
2. 深度重写 (Deep Rewrite)：从简历中挑选 2-3 条核心项目描述，基于 STAR 法则和提供的【优秀模板】进行对比重写
3. 方案优化建议：针对用户简历中平庸的技术方案，给出更具竞争力的替代方案建议
```

给模型一个思考链路（Chain of Thought），能显著降低遗漏风险。引入 **STAR 法则** 等方法论，是为了确保输出结果的专业性。

Constraints：硬性约束

优化前：

```
请严格按照 JSON 格式输出分析结果
```

优化后：

```
# Constraints
- 必须输出严谨的 JSON 格式
- 严禁虚构简历中不存在的业务背景，但可以基于现有背景建议合理的量化指标
- 建议必须具有可操作性，提供"原句 vs 优化句"的对比
```

否定约束比肯定指令更有力。 明确“严禁虚构”是为了对齐事实，防止模型开启“脑补模式”。

Output Format：结构化输出

优化前：

```
请严格按照 JSON 格式输出分析结果
```

优化后：

```
# Output Format
请直接输出一个 JSON 对象，不要包含 Markdown 代码块标签（如 ```json ）。

JSON 结构必须严格包含以下字段：
1. overallScore: 整数，总分（0-100）。
2. scoreDetail: 一个对象，包含以下五个整数字段：
   - projectScore: 项目经验评分（0-40分）
   - skillMatchScore: 技能匹配度评分（0-20）
   - contentScore: 内容完整性评分（0-15）
   - structureScore: 结构清晰度评分（0-15）
   - expressionScore: 表达专业性评分（0-10）
3. summary: 字符串，一句话总结简历的整体情况。
4. strengths: 字符串数组，列出简历的优势点。
5. suggestions: 对象数组，每个对象包含以下字段：
   - category: 建议类别（内容/格式/技能/项目）
   - priority: 优先级（高/中/低）
   - issue: 问题描述
   - recommendation: 具体改进建议
```

字段命名遵循 **camelCase**（小驼峰），与后端 `BeanOutputConverter` 反序列化用的 DTO 字段一致；系统提示词中的 Rubrics 名称（如 projectTechDepth）与 scoreDetail 内字段名（如 projectScore）的对应关系在提示词中写清，便于模型输出稳定。

效果对比

维度	优化前 (随意版)	优化后 (结构化版)
输出稳定性	格式随机，字段缺失，难以解析。	结构高度一致，可直接进行反序列化。
评分一致性	同样一份简历，多次评分波动剧烈。	标准明确，评分结果具备可复现性。
建议质量	空洞笼统，如“建议多写点项目”。	具体可落地，提供 STAR 模式原句对比。
幻觉风险	容易编造不存在的技能或工作经历。	边界清晰，仅在现有背景下进行优化。

Prompt 设计最佳实践

总结以上分析，提炼出 Prompt 设计的核心原则：

RISEN 框架

模块	说明	优先级
Role (角色)	定义 AI 身份，激活特定领域知识。	**高**
Instructions (指令)	核心任务描述，动词要精准。	**高**
Standards (标准)	评估标准、执行规范、量化指标。	**中**
Examples (示例)	提供 Few-shot 示例（可选，但对复杂逻辑极有效）。	**中**
Negations (否定约束)	明确禁止事项，防止模型跑偏。	**高**

关键技巧

1. **用 Markdown 语法**：使用 `#` 标题、列表和表格，模型对这种层次结构的感知能力极强。
2. **数量约束**：如“给出 3 条建议”，避免模型输出过短（偷懒）或过长（啰嗦）。
3. **类型标注**：在 `Output Format` 中明确字段数据类型（如 `int`, `array`, `string`）。
4. **否定约束**：明确“严禁”、“不要”等禁止项，比“请尽量”更具强制力。
5. **字段对齐**：确保 `Scoring Rubrics` 中的维度名称与 `Output Format` 中的字段名逻辑对应。

与项目实现的对应

本项目的简历分析 Prompt 采用**系统提示词 + 用户提示词**分离，与代码的对应关系如下：

内容	实现位置
系统提示词（Role / Task / Standards / Workflow / Constraints / Output Format）	`app/src/main/resources/prompts/resume-analysis-system.st`
用户提示词（简历内容占位符 `{resumeText}`、技术优化基准表、审计维度）	`app/src/main/resources/prompts/resume-analysis-user.st`
格式指令拼接	业务代码将 `outputConverter.getFormat()` 拼接到系统提示词后传入，见 `ResumeGradingService.analyzeResume()`
结构化解析	`BeanOutputConverter<ResumeAnalysisResponseDTO>` + `StructuredOutputInvoker`，解析为 Java Record 后转为业务对象

其他场景的 Prompt 模板也在同一目录下，按用途命名，例如：

- `interview-question-system.st` / `interview-question-user.st`：面试问题生成（含追问）
- `interview-evaluation-system.st` / `interview-evaluation-user.st`：面试答案分批评估
- `interview-evaluation-summary-*.st`：评估结果二次总结
- `knowledgebase-query-system.st` / `knowledgebase-query-user.st` / `knowledgebase-query-rewrite.st`：知识库 RAG 问答与查询改写

详见 Spring AI 与大模型集成 中的 Prompt 管理一节。

六、结语

好的 Prompt 不是一蹴而就的，而是通过不断测试、复盘、迭代出来的。在生产环境中，**结构化设计**能让你的 AI 应用更稳定、更专业、更具确定性。

记住：你给模型的指令越清晰，模型回馈给你的确定性就越高。